



Pimpri-Chinchwad Education Trust's
Pimpri-Chinchwad College of Engineering
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University (SPPU)
ISO 21001:2018 Certified by TUV

Assignment No 2 : Quick Sort

Name : Yashodhan Prakash Suryawanshi
PRN No. : 125B2F004

Class: T.Y. IT - A
Subject: Design Analysis and Algorithm

Github Link:

<https://github.com/Yasu225153/DAA-Lab-Submissions/tree/main/Assignment%20-%202>

1. Problem Statement

A popular OTT platform, StreamFlix, offers personalized recommendations by sorting movies based on user preferences, such as IMDB rating, release year, or watch time popularity. However, during peak hours, sorting large datasets slows down the system. As a backend engineer, you must:

- Implement Quicksort to efficiently sort movies based on various user-selected parameters.
- Handle large datasets containing of movies while maintaining fast response times

2. Course Objective

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

3. Course Outcome

- Analyze the asymptotic performance of algorithms.
- Solve computational problems by applying suitable paradigms of Divide and Conquer or Greedy methodologies

4. Theory

Quick Sort is a widely used sorting algorithm known for its **efficiency, simplicity, and low memory usage**. It follows the **Divide and Conquer** approach and is suitable for sorting large datasets. Quick Sort is an **in-place sorting algorithm**, as it requires only a small amount of additional memory and does not need a separate merging process.

The working of Quick Sort consists of three main steps:

Divide

A **pivot element** is selected from the array. The pivot can be chosen using different methods, such as the first element, last element, random element, or median-of-three method.

Partition

The array is rearranged so that elements **smaller than the pivot** are placed on the left, while elements **greater than the pivot** are placed on the right. The pivot is then placed in its correct position.

Conquer

Quick Sort is recursively applied to the left and right subarrays until the complete array becomes sorted.

Quick Sort has an **average-case time complexity of $O(n \log n)$** . However, if the pivot selection results in highly unbalanced partitions, the worst-case complexity can become **$O(n^2)$** .

To improve its performance, techniques such as **random pivot selection, median-of-three, tail recursion elimination, and using Insertion Sort for small subarrays** can be applied.

5. Time and Space Complexity

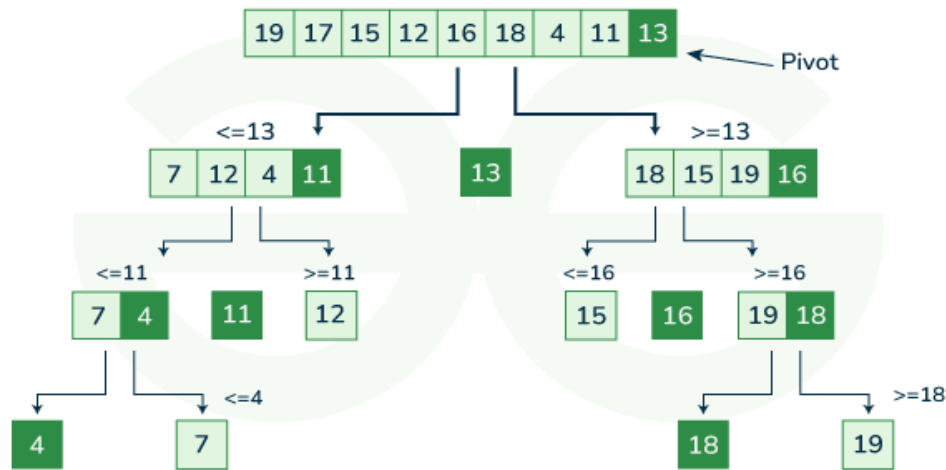
Case	Time Complexity	Description
Best Case	$O(n \log n)$	Balanced partitions
Average Case	$O(n \log n)$	Typical for random input
Worst Case	$O(n^2)$	When pivot is poorly chosen
Space Complexity	$O(n \log n)$	For recursive stack calls

Quick Sort Algorithm

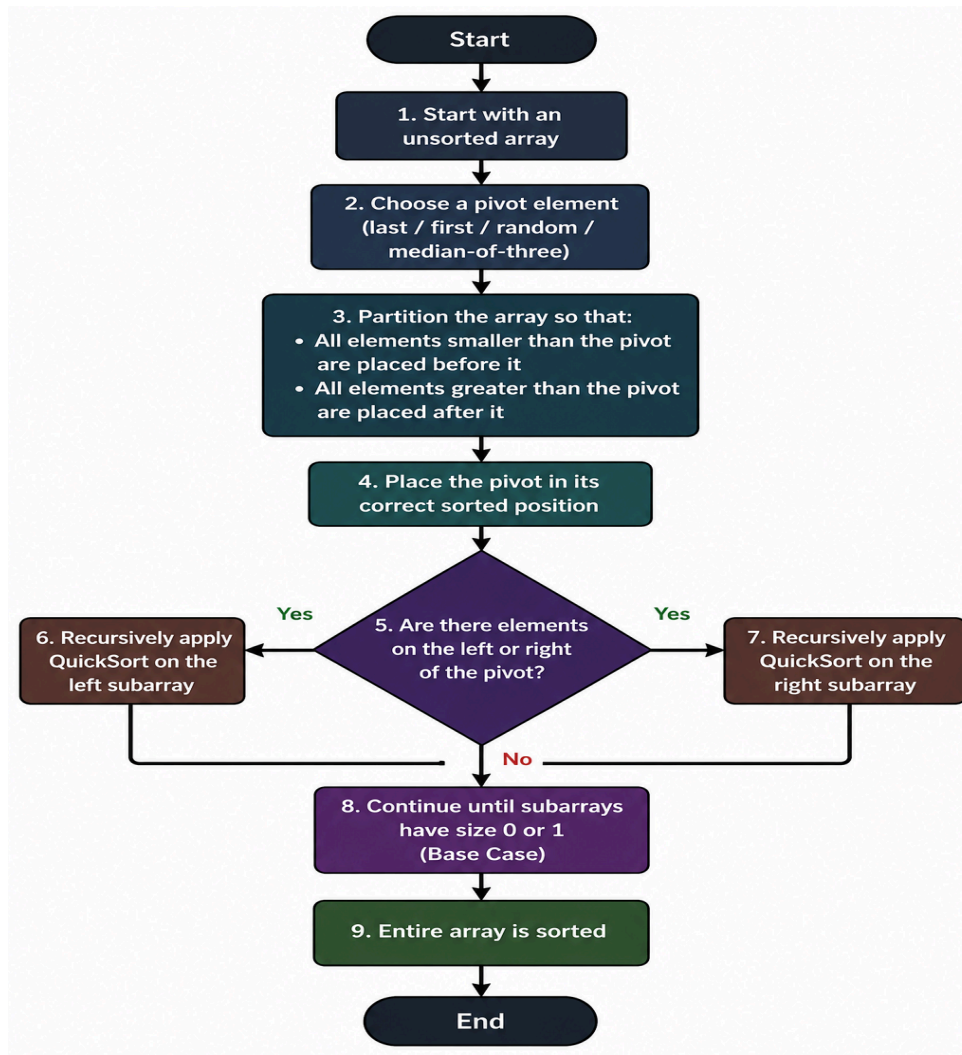
1. Start with the unsorted array.
2. Select a pivot element from the array.
3. Partition the array by placing elements smaller than the pivot on its left and larger elements on its right.
4. Place the pivot in its correct sorted position.
5. Recursively apply Quick Sort to the left subarray.
6. Recursively apply Quick Sort to the right subarray.

7. Continue the process until each subarray contains zero or one element.
8. The complete array is then obtained in sorted order.

Quick Sort Algorithm



Quick Sort Algorithm



Pseudo code

```
ALGORITHM QuickSort(arr, low, high)
```

```
1.  if low < high then
2.      pivotIndex ← Partition(arr, low, high)

3.      // Sort elements before the pivot
4.      QuickSort(arr, low, pivotIndex - 1)

5.      // Sort elements after the pivot
6.      QuickSort(arr, pivotIndex + 1, high)
7.  end if
```

```
END ALGORITHM
```

```
ALGORITHM Partition(arr, low, high)
```

```
1.  pivot ← arr[high]           // Choose last element as pivot
2.  i ← low - 1                 // Index of smaller element

3.  for j ← low to high - 1 do
4.      if arr[j] ≤ pivot then
5.          i ← i + 1
6.          SWAP(arr[i], arr[j])
7.      end if
8.  end for

9.  SWAP(arr[i + 1], arr[high]) // Place pivot correctly
10. return i + 1               // Return pivot index
```

```
END ALGORITHM
```

6. Example

Given Array:

[3, 1, 4, 2]

Step 1: Choose Pivot

Select the **last element** as the pivot.

Pivot = 2

Step 2: Partition the Array

Compare each element with the pivot:

- $3 > 2 \rightarrow$ placed on the right
- $1 \leq 2 \rightarrow$ placed on the left
- $4 > 2 \rightarrow$ placed on the right

After partitioning:

[1, 2, 4, 3]

↑

Pivot

The pivot 2 is now in its correct position.

Step 3: Recursive Calls

The array is divided into:

- **Left:** [1]
- **Right:** [4, 3]

The left subarray [1] has one element, so it is already sorted.

Step 4: Quick Sort on [4, 3]

Choose the last element as the pivot.

Pivot = 3

- $4 > 3 \rightarrow$ placed on the right

After partitioning:

[3, 4]

Final Sorted Array

[1, 2, 3, 4]

Therefore, the final sorted array is:

[1, 2, 3, 4]

7. Code

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

struct Movie {
    string title;
    double rating;
    int releaseYear;
    int popularity;
};

// Compare movies
bool compareMovies(const Movie& a, const Movie& b, int choice) {
    if (choice == 1)
        return a.rating > b.rating;
    else if (choice == 2)
        return a.releaseYear > b.releaseYear;
    else
        return a.popularity > b.popularity;
}

// Partition Function
int partition(vector<Movie>& movies, int low, int high, int choice) {
    Movie pivot = movies[high];

    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (compareMovies(movies[j], pivot, choice)) {
            i++;
            swap(movies[i], movies[j]);
        }
    }

    swap(movies[i + 1], movies[high]);

    return i + 1;
}

// QuickSort Function
void quickSort(vector<Movie>& movies, int low, int high, int choice) {
```

```

    if (low < high) {
        int pivotIndex = partition(movies, low, high, choice);

        quickSort(movies, low, pivotIndex - 1, choice);
        quickSort(movies, pivotIndex + 1, high, choice);
    }
}

// Display first 10 records
void displayMovies(const vector<Movie>& movies) {
    cout << "\n-----\n";
    cout << "Title\t\tRating\tYear\tPopularity\n";
    cout << "-----\n";

    int limit = min(10, (int)movies.size());

    for (int i = 0; i < limit; i++) {
        cout << movies[i].title << "\t"
              << movies[i].rating << "\t"
              << movies[i].releaseYear << "\t"
              << movies[i].popularity << endl;
    }
}

int main() {

    vector<Movie> movies;

    // Read CSV Records
    ifstream file("movies_real_titles .csv");

    if (!file.is_open()) {
        cout << "Error: Could not open movies_real_titles.csv" << endl;
        return 1;
    }

    string line;

    // Skip header
    getline(file, line);

    while (getline(file, line)) {

        if (line.empty())
            continue;

        stringstream ss(line);

        Movie movie;

        string rating;
        string year;
        string popularity;

```

```

        getline(ss, movie.title, ',');
        getline(ss, rating, ',');
        getline(ss, year, ',');
        getline(ss, popularity);

        movie.rating = stod(rating);
        movie.releaseYear = stoi(year);
        movie.popularity = stoi(popularity);

        movies.push_back(movie);
    }

    file.close();

    cout << "Total movies loaded: " << movies.size() << endl;

    if (movies.empty()) {
        cout << "No movie records found." << endl;
        return 1;
    }

    // Select Sorting Parameter
    int choice;

    cout << "\n===== STREAMFLIX MOVIE SORTING =====\n";

    cout << "\nSelect sorting parameter:\n";
    cout << "1. IMDB Rating\n";
    cout << "2. Release Year\n";
    cout << "3. Popularity\n";

    cout << "\nEnter your choice: ";
    cin >> choice;

    if (choice < 1 || choice > 3) {
        cout << "Invalid choice!" << endl;
        return 1;
    }

    // Apply QuickSort
    quickSort(movies, 0, movies.size() - 1, choice);

    cout << "\nSorted Movie Data:\n";

    if (choice == 1)
        cout << "(Highest IMDB Rating First)\n";
    else if (choice == 2)
        cout << "(Latest Release Year First)\n";
    else
        cout << "(Highest Popularity First)\n";

    // Display only 10 records

```



```

displayMovies(movies);

// Create Output CSV
ofstream output("movies_sorted_output.csv");

if (!output.is_open()) {
    cout << "Error: Could not create output file." << endl;
    return 1;
}

output << "title,rating,release_year,popularity\n";

for (const Movie& movie : movies) {
    output << movie.title << ","
        << movie.rating << ","
        << movie.releaseYear << ","
        << movie.popularity << "\n";
}

output.close();

cout << "\n-----\n";
cout << "Sorting completed successfully!\n";
cout << "First 10 sorted records displayed above.\n";
cout << "Complete sorted data saved in: movies_sorted_output.csv\n";

return 0;
}

```

8. Output

Example Dataset – Movies Sorted by IMDb Rating

Given Movie Ratings:

[8.2, 6.5, 9.0, 7.3, 8.7]

Step 1: Choose Pivot

Select the **last element** as the pivot.

Pivot = 8.7

Partition the elements around the pivot:

- Elements $\leq 8.7 \rightarrow [8.2, 6.5, 7.3]$
- Elements $> 8.7 \rightarrow [9.0]$

After partitioning:

[8.2, 6.5, 7.3, 8.7, 9.0]

↑

Pivot

The pivot 8.7 is placed in its correct position.

Step 2: Apply Quick Sort Recursively

Left subarray: [8.2, 6.5, 7.3]

Choose the last element as the pivot.

Pivot = 7.3

- Elements $\leq 7.3 \rightarrow [6.5]$
- Elements $> 7.3 \rightarrow [8.2]$

After partitioning:

[6.5, 7.3, 8.2]

Right subarray: [9.0]

Since it contains only one element, it is already sorted.

Step 3: Combine Results

The sorted parts are:

[6.5, 7.3, 8.2] + [8.7] + [9.0]

Final Output

Movies sorted in ascending order of IMDb rating:[6.5, 7.3, 8.2, 8.7, 9.0]

8. Conclusion

Quick Sort is an efficient and memory-friendly algorithm for sorting large movie datasets based on parameters such as **IMDb rating, release year, or popularity**. Its fast average-case performance and low memory requirement make it suitable for applications that require quick and responsive sorting.